# AI CHAKRAVYUH

# 50+ Page Detailed Implementation Manual

Final Round: AI Tool-Call Checkpoint | Team GOAT Coders

This manual explains what to build, who builds it, how to test it, how to merge it and how to demonstrate it without conflicts.

Use synthetic data and simulated tools only. Never connect real student records or perform real data exfiltration.

# 02. Read the Challenge as Acceptance Criteria

**Purpose:** Translate every sentence of the challenge into a testable requirement.

**Work through this page in order.**

## Step-by-step procedure

• Write the six mandatory capabilities on the team board: append-only log, hash verification, deterministic replay, MCP proxy, approval codes and live scenarios.

• Add one acceptance test for every capability before coding.

• Mark Airtable/Sheets as removed from the security path. The log is now the source of truth.

• Mark the AI role clearly: interpretation and explanation, not execution authority.

• Agree that all tool execution uses simulated tools in the hackathon demo.

## Definition of done and tests

• The board contains an owner, file location and test for each requirement.

• No requirement is described only with words such as secure or intelligent; each has observable evidence.

# 03. Freeze Roles and Communication

**Purpose:** Prevent duplicate work and conflicting design decisions.

**Work through this page in order.**

## Step-by-step procedure

• Name Member 1 the integration coordinator and schema owner.

• Name Members 2-4 owners of proxy, AI and approval/UI respectively.

• Create a shared chat message with the frozen contracts in this manual.

• Use one daily 10-minute sync: completed, next, blocker.

• Never silently change a field used by another member; open an issue first.

• Each member keeps a short CHANGELOG in their module README.

## Definition of done and tests

• Every member can explain their input, output and tests.

• All cross-module questions are answered before parallel coding begins.

# 04. Install the Development Environment

**Purpose:** Create one reproducible local environment for all members.

**Work through this page in order.**

## Step-by-step procedure

• Install Python 3.11 or the version agreed by the team.

• Install Git and verify it is available in the terminal.

• Create a virtual environment named .venv.

• Install reportlab only for this guide; install project dependencies from requirements.txt.

• Create a .env.example but never commit real tokens or email credentials.

• Run the smoke test before writing feature code.

## Commands or reference

```
python -m venv .venv .venv\Scripts\activate pip install -r requirements.txt python --version pytest -q
```

## Definition of done and tests

• All four members run the same Python version.

• The repository opens and tests can run from a clean clone.

# 05. Create the Repository

**Purpose:** Set up the repository so ownership is visible.

**Work through this page in order.**

## Step-by-step procedure

• Create a private or public GitHub repository named ai-chakravyuh.

• Clone it on every member machine.

• Create main and integration branches.

• Add .gitignore for .venv, .env, __pycache__, generated logs and secrets.

• Add README.md with project purpose, safety limitations and setup.

• Protect main from direct pushes if repository settings allow it.

## Commands or reference

```
git clone git checkout -b integration git push -u origin integration
```

## Definition of done and tests

• A clean clone contains no secrets.

• main and integration exist.

• The README tells a new person how to run tests.

# 06. Create the Folder Structure

**Purpose:** Create stable module boundaries before implementation.

**Work through this page in order.**

## Step-by-step procedure

• Create core, proxy, ai, approval, ui, scenarios, data, tests and docs.

• Add __init__.py to Python packages.

• Put only synthetic fixtures under data.

• Put shared schemas in core/schemas.py.

• Put integration tests in tests/integration after unit tests are stable.

• Do not place business logic in UI files.

## Commands or reference

```
ai-chakravyuh/ core/ proxy/ ai/ approval/ ui/ scenarios/ data/ tests/ docs/
```

## Definition of done and tests

• Each member has an owned folder.

• No two members need to edit the same implementation file.

# 07. Freeze the Naming Convention

**Purpose:** Avoid the most common integration failure: inconsistent names.

**Work through this page in order.**

## Step-by-step procedure

• Use snake_case for Python variables and JSON fields.

• Use request_id for the original tool request identity.

• Use evidence_id for the security decision evidence identity.

• Use event_id for an append-only log event identity.

• Use scenario_id for a repeatable test scenario.

• Use decision values exactly as uppercase ALLOW, HOLD and DENY.

• Use ISO-8601 UTC timestamps for display, but never use current time in replay decisions.

## Definition of done and tests

• A sample JSON request validates against the same schema in all four modules.

# 08. Shared Tool Request Contract

**Purpose:** Define the object that moves through every module.

**Work through this page in order.**

## Step-by-step procedure

• Create a Python dataclass or validation model named ToolRequest.

• Require request_id, tool, arguments, purpose and scenario_id.

• Canonicalise arguments before hashing or approval binding.

• Reject missing or unknown critical fields at the proxy boundary.

• Keep arguments JSON-serialisable so MCP and the log can reproduce them.

• Use synthetic filenames and recipients in fixtures.

## Commands or reference

```
{"request_id":"req-001","tool":"send_email","arguments":{"recipient":"external@example.com","file":"synth
etic_records.pdf"},"purpose":"Send report","scenario_id":"malicious-pdf-001"}
```

## Definition of done and tests

• A request serialises to JSON and deserialises without losing fields.

• Changing one argument changes the request fingerprint.

# 09. Shared Decision Contract

**Purpose:** Define exactly what the checkpoint returns.

**Work through this page in order.**

## Step-by-step procedure

• Create a Decision object with request_id, decision, evidence_id and reason.

• Include risk signals and policy_version when available.

• Require evidence_id for DENY and HOLD.

• For ALLOW, still create evidence so the safe path is auditable.

• Make reason human-readable but keep policy facts structured.

• Never let free-form AI text replace the decision field.

## Commands or reference

```
{"request_id":"req-001","decision":"DENY","evidence_id":"ev-001","reason":"Unauthorised external
transfer","policy_version":"v1"}
```

## Definition of done and tests

• The proxy can render every decision without knowing AI internals.

# 10. Shared Log Event Contract

**Purpose:** Define the tamper-evident event shape.

**Work through this page in order.**

## Step-by-step procedure

• Create event_id, sequence, timestamp, event_type, request_id and payload.

• Add previous_hash and entry_hash after the payload is canonicalised.

• Use one event per line in data/security_log.jsonl.

• Record input received, AI analysis, layer result, checkpoint decision, approval attempt and tool result.

• Do not store secrets or raw approval codes in the log.

• Store evidence IDs so the UI can link all related events.

## Commands or reference

```
{"sequence":12,"event_id":"evt-012","event_type":"CHECKPOINT_DECISION","request_id":"req-001","decision":
"DENY","evidence_id":"ev-001","payload":{},"previous_hash":"...","entry_hash":"..."}
```

## Definition of done and tests

• Every event can be understood without reading another database.

# 11. Member 1: Design the Hash Chain

**Purpose:** Choose a simple algorithm the jury can understand.

**Work through this page in order.**

## Step-by-step procedure

• Set GENESIS = a fixed visible string such as GENESIS_AI_CHAKRAVYUH_V1.

• For sequence one, previous_hash equals SHA-256(GENESIS).

• For later entries, previous_hash equals the prior entry_hash.

• Build a copy of the event without entry_hash.

• Serialise with sorted keys and compact separators.

• Set entry_hash to SHA-256(previous_hash + canonical_event).

• Document the formula in core/README.md.

## Definition of done and tests

• The same event produces the same hash on every machine.

• Changing whitespace in the JSON does not change the canonical hash.

# 12. Member 1: Implement Append

**Purpose:** Implement append without allowing old entries to be overwritten.

**Work through this page in order.**

## Step-by-step procedure

• Open the log in append mode only.

• Read the final entry to obtain its hash, or use GENESIS for an empty file.

• Assign sequence as final sequence plus one.

• Create event_id with a UUID, but record it before hashing.

• Write exactly one JSON object followed by a newline.

• Flush the file after writing so the demo can immediately read it.

• Return the complete event including entry_hash.

## Definition of done and tests

• Appending two events creates two lines.

• The second previous_hash equals the first entry_hash.

# 13. Member 1: Implement Verification

**Purpose:** Make tamper detection precise and easy to demonstrate.

**Work through this page in order.**

## Step-by-step procedure

• Read the file line by line and parse each JSON object.

• Check sequence starts at one and increments by one.

• Check each previous_hash against the prior computed entry_hash.

• Recompute the current hash from the canonical event.

• Stop or collect errors with line number, sequence and event_id.

• Return a JSON result with valid=false and affected_entry.

• Use a non-zero process exit code when invalid.

## Commands or reference

```
python -m core.verify data/security_log.jsonl
```

## Definition of done and tests

• Changing a reason identifies the changed event.

• Deleting a line identifies the next broken sequence/hash relationship.

# 14. Member 1: Implement Replay

**Purpose:** Make the security result reproducible without calling the model again.

**Work through this page in order.**

## Step-by-step procedure

• Replay only recorded scenario inputs, analysis outputs and policy_version.

• Do not call a live LLM during replay.

• Do not use current time, random numbers or external network state.

• Run the deterministic policy function for each checkpoint event.

• Compare produced decisions with recorded decisions.

• Output a decision_digest so two runs can be compared.

## Commands or reference

```
python -m core.replay data/security_log.jsonl --output replay.json
```

## Definition of done and tests

• Replay twice produces the same decision_digest.

• A replay mismatch identifies request_id and expected/observed decision.

# 15. Member 1: Test the Log Thoroughly

**Purpose:** Finish the first independent module before integration.

**Work through this page in order.**

## Step-by-step procedure

• Write fixtures for empty, one-entry and multi-entry logs.

• Test valid append and valid verification.

• Copy a valid log and modify one payload value.

• Copy a valid log and modify one entry_hash.

• Copy a valid log and remove one middle line.

• Copy a valid log and swap two lines.

• Run replay with a fixed fixture twice.

## Definition of done and tests

• pytest tests/test_log.py passes.

• The test output demonstrates exact tamper location rather than only saying invalid.

# 16. Member 2: Understand MCP and JSON-RPC

**Purpose:** Keep the transport narrow and testable.

**Work through this page in order.**

## Step-by-step procedure

• Treat stdin as a stream of JSON-RPC request lines.

• Treat stdout as a stream of JSON-RPC response lines.

• Use stderr for debug messages.

• Support method, params, id and jsonrpc fields.

• Return JSON-RPC errors for invalid requests.

• Keep the simulated tool executor behind a Python function so tests can replace it.

• Document that direct calls bypassing the proxy are unsupported.

## Definition of done and tests

• A raw request can be piped to the server and produces one valid response line.

# 17. Member 2: Validate Requests

**Purpose:** Reject malformed or unsafe requests before policy evaluation.

**Work through this page in order.**

## Step-by-step procedure

• Parse JSON and catch decode errors.

• Check jsonrpc equals 2.0.

• Check id exists and is stable.

• Check method is the supported tool-call method.

• Check params contains a tool request.

• Reject non-object arguments.

• Return an error without executing anything.

## Definition of done and tests

• Malformed JSON, missing method and invalid arguments each produce a deterministic error.

# 18. Member 2: Build the Checkpoint

**Purpose:** Make the decision logic external to the AI.

**Work through this page in order.**

## Step-by-step procedure

• Create a PolicyDecisionEngine class.

• Evaluate allowlisted tool names first.

• Evaluate resource and destination restrictions next.

• Evaluate AI risk signals as inputs, not commands.

• Apply approval rules for high-risk but reviewable actions.

• Return ALLOW, HOLD or DENY with evidence_id and reason.

• Write the decision event through the log interface.

## Definition of done and tests

• Identical request and policy_version always produce identical decisions.

# 19. Member 2: Implement ALLOW

**Purpose:** Make safe execution observable.

**Work through this page in order.**

## Step-by-step procedure

• Check the decision equals ALLOW before calling the executor.

• Pass only the validated tool and arguments.

• Record tool_started and tool_completed events.

• Return the simulated tool response to the MCP client.

• Use a safe tool such as summarize_document for the benign demo.

• Never use a real email, database or filesystem export tool.

## Definition of done and tests

• The mock executor records one call for an allowed request.

• The log contains the decision and tool result.

# 20. Member 2: Implement DENY

**Purpose:** Make denial final and evidence-linked.

**Work through this page in order.**

## Step-by-step procedure

• Do not call the executor when decision is DENY.

• Create a JSON-RPC error response.

• Include evidence_id in the error data.

• Include a short reason without exposing sensitive content.

• Append the denial before returning the response.

• Make repeated identical attempts produce separate event IDs but the same policy result.

## Definition of done and tests

• A spy executor confirms zero calls for denied requests.

# 21. Member 2: Implement HOLD Concurrency

**Purpose:** Hold only what needs approval.

**Work through this page in order.**

## Step-by-step procedure

• Store pending requests by request_id.

• Return a HOLD response immediately to that request or maintain its pending state.

• Do not block the entire server loop.

• Allow an independent safe request to be processed.

• Require the approval service to release the exact pending request.

• Remove the pending record after release, expiry or denial.

## Definition of done and tests

• A held request and an independent allowed request can be tested in one run.

# 22. Member 2: Test the Proxy

**Purpose:** Complete proxy tests before connecting the AI module.

**Work through this page in order.**

## Step-by-step procedure

• Test valid JSON-RPC parsing.

• Test invalid JSON-RPC errors.

• Test ALLOW reaches executor.

• Test DENY does not reach executor.

• Test HOLD does not block an independent request.

• Test evidence_id appears in DENY response.

• Test all proxy events are logged.

## Definition of done and tests

• pytest tests/test_proxy.py passes with a fake executor and fake log.

# 23. Member 3: Define AI Responsibilities

**Purpose:** Use AI only where semantic interpretation adds value.

**Work through this page in order.**

## Step-by-step procedure

• Define prompt_injection as an attempt by content to override system/user intent.

• Define goal_drift as a mismatch between original purpose and proposed action.

• Define sensitive_data_risk as semantic exposure risk.

• Return structured signals and a concise explanation.

• Keep recommended_action advisory.

• Version the analyser prompt/model configuration.

• Record the analyser version in the log.

## Definition of done and tests

• The AI module can be replaced by a mock without changing proxy code.

# 24. Member 3: Build the Malicious PDF Fixture

**Purpose:** Create a safe synthetic attack input.

**Work through this page in order.**

## Step-by-step procedure

• Create scenarios/malicious_pdf.json.

• Set original_goal to summarise a college circular.

• Put an embedded instruction that asks to ignore the goal.

• Ask it to access synthetic student records and send them externally.

• Include expected signals and expected final policy result.

• Never include real names, IDs, emails or records.

• Keep the fixture small enough for the live demo.

## Definition of done and tests

• The scenario is readable by a judge and repeatable by the test suite.

# 25. Member 3: Build the Safe Fixture

**Purpose:** Prove that the system is not just blocking everything.

**Work through this page in order.**

## Step-by-step procedure

• Create scenarios/safe_summary.json.

• Set original_goal to summarise a public synthetic notice.

• Use a benign document without override instructions.

• Request only summarize_document.

• Set expected prompt_injection=false and goal_drift=false.

• Set expected final decision ALLOW.

• Add a second benign edge case if time permits.

## Definition of done and tests

• The safe request reaches the mock tool and is logged as ALLOW.

# 26. Member 3: Implement Goal Drift

**Purpose:** Compare meaning, not only keywords.

**Work through this page in order.**

## Step-by-step procedure

• Accept original_goal and proposed_plan as inputs.

• Represent the original goal as a short normalized intent.

• Represent the proposed plan as actions and targets.

• Flag a mismatch when the proposed target or action is outside scope.

• Return evidence phrases that explain the mismatch.

• Use deterministic mock output for unit tests.

• Pass the signal to the policy engine.

## Definition of done and tests

• Summarisation to external export is flagged even if no exact attack phrase is present.

# 27. Member 3: Implement Prompt-Injection Analysis

**Purpose:** Detect instructions hidden in untrusted content.

**Work through this page in order.**

## Step-by-step procedure

- Label the source as untrusted_document.
- Extract instruction-like sentences separately from data text.
- Identify override requests such as ignore previous task.
- Identify requests for secrets, permissions or external transfer.
- Return source span or short evidence text for the report.
- Do not execute or follow extracted instructions.
- Let DWAAR and SANKALP consume the result.

## Definition of done and tests

- The malicious fixture produces a clear explanation suitable for the jury.

# 28. Member 3: Implement Sensitive-Data Risk

**Purpose:** Detect meaning and context beyond keywords.

**Work through this page in order.**

## Step-by-step procedure

• Define synthetic categories: student_record, identity_data and confidential_report.

• Check requested data against the scenario purpose.

• Flag unnecessary data movement.

• Flag external destinations.

• Return low, medium or high risk.

• Ensure the UI does not display raw sensitive data.

• Log only metadata and evidence references.

## Definition of done and tests

• A request for synthetic records to an external recipient is high risk.

# 29. Member 3: Test AI Signals

**Purpose:** Make model variability safe for development.

**Work through this page in order.**

## Step-by-step procedure

• Use a MockAnalyser with fixed results for unit tests.

• Test malicious document signals.

• Test safe document signals.

• Test goal drift with different action names.

• Test structured output validation.

• Test analyser failure returns a safe fallback such as HOLD.

• Test that AI output cannot directly invoke a tool.

## Definition of done and tests

• pytest tests/test_ai_analysis.py passes without network access.

# 30. Member 4: Design Approval Binding

**Purpose:** Approval must authorize one exact action only.

**Work through this page in order.**

## Step-by-step procedure

• Define approval_id and request_id.

• Canonicalise tool name and arguments.

• Create action_digest = SHA-256(canonical tool request).

• Bind the approval to action_digest.

• Set short expiry for the demo.

• Store code hash, not the plain code, in the service.

• Record created, sent, verified, rejected and expired events.

## Definition of done and tests

• Changing one argument makes the approval invalid.

# 31. Member 4: Generate and Send Codes

**Purpose:** Move approval outside the web UI.

**Work through this page in order.**

## Step-by-step procedure

• Generate a cryptographically random numeric or alphanumeric code.

• Send it through a configured Telegram/email/SMS adapter.

• Use a mock notification adapter for tests.

• Show only masked destination in the UI.

• Never put the code in the browser URL or frontend source.

• Log notification status without logging the code.

## Definition of done and tests

• The demo can show that the code arrives outside the dashboard.

# 32. Member 4: Verify One-Time Codes

**Purpose:** Handle every failure explicitly.

**Work through this page in order.**

## Step-by-step procedure

• Look up the pending approval by approval_id.

• Recompute action_digest from the current request.

• Compare the provided code hash.

• Check expiry before accepting.

• Check consumed status before accepting.

• Mark consumed atomically before releasing the request.

• Return approval evidence_id and log the result.

## Definition of done and tests

• Correct code succeeds once. Wrong, expired, reused and mismatched codes are denied.

# 33. Member 4: Build the Dashboard

**Purpose:** Show evidence rather than decorative screens.

**Work through this page in order.**

## Step-by-step procedure

• Add scenario selector.

• Add Run Scenario button.

• Show the original goal and synthetic input summary.

• Show seven layer cards with status and reason.

• Show tool-call timeline and decision.

• Show evidence ID and log sequence.

• Add Verify Log and Replay buttons.

• Show approval status without exposing codes.

## Definition of done and tests

• A judge can understand the attack path without reading source code.

# 34. Member 4: Test UI and Approval

**Purpose:** Keep UI tests focused on behaviour.

**Work through this page in order.**

## Step-by-step procedure

• Test rendering of ALLOW, HOLD and DENY states.

• Test evidence ID is clickable or searchable in the log view.

• Test retry does not duplicate approval consumption.

• Test error messages are actionable.

• Test UI cannot submit a different action with the same approval.

• Test the dashboard works with fixture responses.

## Definition of done and tests

• pytest tests/test_approval.py and the manual UI checklist pass.

# 35. Create the Policy Matrix

**Purpose:** Turn security responsibilities into explicit decisions.

**Work through this page in order.**

## Step-by-step procedure

• Create a policy matrix in core/policies.py or a versioned JSON file.

• For each layer define input, condition, output and evidence.

• Define which conditions are DENY versus HOLD.

• Define safe tools and safe destinations.

• Define approval-required tools and data categories.

• Assign policy_version=v1.

• Do not put hidden policy decisions in UI code.

| Condition | Decision |
|---|---|
| Unknown tool | DENY |
| External transfer with sensitive data | DENY or HOLD by policy |
| High-risk reviewable action | HOLD |
| Approved summarize_document on synthetic public text | ALLOW |

## Definition of done and tests

• The same request evaluated twice with v1 gives the same decision.

# 36. Wire the Seven Layers

**Purpose:** Represent layer results in the event trail.

**Work through this page in order.**

## Step-by-step procedure

• Create LayerResult with layer_id, status, reason and evidence_id.

• Run DWAAR on content trust.

• Run PEHCHAAN on identity and scope.

• Run SMRITI on provenance.

• Run SANKALP on goal alignment.

• Run ASTRA on tool safety.

• Run MARYADA on approval requirement.

• Let DRISHTI record and contain the workflow.

• The final checkpoint decision remains one deterministic result.

## Definition of done and tests

• The UI can show every layer even when the final decision is ALLOW.

# 37. Connect AI to Policy

**Purpose:** Use AI output safely.

**Work through this page in order.**

## Step-by-step procedure

• Send ToolRequest and original_goal to the analyser.

• Validate the returned JSON schema.

• If the analyser fails, use conservative HOLD for high-impact calls.

• Pass only validated signals to PolicyDecisionEngine.

• Never concatenate AI text into executable code.

• Record the AI result and model_version before checkpoint decision.

• Make final policy thresholds visible in docs.

## Definition of done and tests

• A malicious AI recommendation cannot bypass an explicit DENY rule.

# 38. Connect Proxy to Log

**Purpose:** Make the log the communication trail.

**Work through this page in order.**

## Step-by-step procedure

• Inject a LogWriter interface into the proxy.

• Log request_received before analysis.

• Log layer_evaluated for each layer.

• Log checkpoint_decision before execution or hold.

• Log tool_completed only after simulated execution.

• Log approval events for HOLD.

• Use the same request_id for all events.

## Definition of done and tests

• One request produces a trace that can be followed from first event to final result.

# 39. Connect Approval to Proxy

**Purpose:** Release only the correct pending call.

**Work through this page in order.**

## Step-by-step procedure

• Proxy creates a pending record for HOLD.

• Approval service receives request_id and action_digest.

• Notification service sends the one-time code.

• Verification service checks exact binding.

• Proxy releases the matching pending request only.

• Proxy re-evaluates expiry and policy before execution.

• Proxy logs release or denial.

## Definition of done and tests

• A code for send_email cannot release export_records or a changed recipient.

# 40. Add Deterministic Replay Fixtures

**Purpose:** Prepare the proof that replay works.

**Work through this page in order.**

## Step-by-step procedure

• Run malicious and safe scenarios once with recorded analyser outputs.

• Save the original log as data/fixtures/original_log.jsonl.

• Save the decision list expected_decisions.json.

• Run replay without network access.

• Compare decision list and digest.

• Document that replay does not regenerate AI analysis.

• Keep the original fixture unchanged during the demo.

## Definition of done and tests

• Replay output equals expected decisions exactly.

# 41. Run Unit Tests

**Purpose:** Check each owner module independently.

**Work through this page in order.**

## Step-by-step procedure

• Member 1 runs test_log.py.

• Member 2 runs test_proxy.py.

• Member 3 runs test_ai_analysis.py.

• Member 4 runs test_approval.py.

• Fix failures in the owning branch first.

• Do not merge a failing branch to integration.

• Save a test output screenshot or terminal transcript for the jury folder.

## Commands or reference

```
pytest tests/test_log.py tests/test_proxy.py tests/test_ai_analysis.py tests/test_approval.py -q
```

## Definition of done and tests

• All unit tests pass on a clean environment.

# 42. Merge Member 1 First

**Purpose:** Integrate the shared foundation with minimal conflict.

**Work through this page in order.**

## Step-by-step procedure

• Member 1 opens PR into integration.

• Everyone reviews schemas and hash algorithm.

• Merge only after log tests pass.

• Tag the merge commit foundation-v1.

• Other members pull integration.

• Do not squash away useful test commits if the team wants traceability.

## Definition of done and tests

• All other branches can import core.schemas and core.append_log.

# 43. Merge Member 2 Second

**Purpose:** Integrate the checkpoint against the real log.

**Work through this page in order.**

## Step-by-step procedure

• Member 2 rebases or merges latest integration into feature branch.

• Replace fake log calls with the stable LogWriter interface.

• Run proxy tests and integration smoke tests.

• Open PR with sample JSON-RPC request and response.

• Review that stdout stays JSON-only.

• Merge into integration and tag proxy-v1.

## Definition of done and tests

• A tool call cannot reach the executor without a decision.

# 44. Merge Member 3 Third

**Purpose:** Connect semantic analysis without changing enforcement.

**Work through this page in order.**

## Step-by-step procedure

• Member 3 pulls integration.

• Adapt analyser output to the frozen DecisionSignals schema.

• Add malicious and safe scenarios.

• Use mock analyser in CI tests.

• Run the end-to-end policy test.

• Review that AI output is advisory and logged.

• Merge and tag ai-v1.

## Definition of done and tests

• Malicious PDF is blocked/held and safe request remains allowed.

# 45. Merge Member 4 Fourth

**Purpose:** Connect approval and UI after backend behaviour is stable.

**Work through this page in order.**

## Step-by-step procedure

• Member 4 pulls integration.

• Connect UI to proxy APIs or local service adapter.

• Connect approval adapter to HOLD queue.

• Use mock notification in automated tests and configured external channel in demo.

• Run approval tests and UI smoke test.

• Merge and tag demo-v1.

## Definition of done and tests

• One-time approval works end to end without exposing codes in the UI.

# 46. Resolve Merge Conflicts Safely

**Purpose:** Use contracts instead of guessing.

**Work through this page in order.**

## Step-by-step procedure

- Stop and read both versions before editing a conflict.
- Keep Member 1 schema names as authoritative.
- Keep each module’s internal implementation in its owner branch.
- Do not resolve by accepting all incoming changes.
- Run targeted tests after each conflict resolution.
- Ask the affected owner to review the resolved file.
- Record any contract change in docs/CHANGELOG.md.

## Definition of done and tests

- No conflict resolution is merged without tests.

# 47. End-to-End Test A: Malicious PDF

**Purpose:** Demonstrate the primary security story.

**Work through this page in order.**

## Step-by-step procedure

- Start the log with a clean copy or record a scenario_start event.
- Submit malicious_pdf scenario.
- Show DWAAR marks content untrusted.
- Show SANKALP detects task drift.
- Show ASTRA denies external transfer or MARYADA holds it.
- Show the MCP error contains evidence_id if DENY.
- Show log events in order.
- Run verify after the scenario.

## Definition of done and tests

- Expected result is not merely a red screen; it includes decision, reason, evidence and log trace.

# 48. End-to-End Test B: Safe Request

**Purpose:** Prove usability and avoid false positives.

**Work through this page in order.**

## Step-by-step procedure

- Submit safe_summary scenario.
- Show no prompt injection and no goal drift.
- Show least-privilege tool is allowed.
- Show MCP proxy invokes mock summarise tool.
- Show tool result and log event.
- Run verify and replay.
- Compare the result with expected ALLOW.

## Definition of done and tests

- The system allows a legitimate request and records it completely.

# 49. End-to-End Test C: Approval

**Purpose:** Demonstrate exact, expiring human control.

**Work through this page in order.**

## Step-by-step procedure

• Submit approval_transfer scenario.

• Show checkpoint returns HOLD only for the transfer request.

• Show unrelated safe request continues.

• Show code arrives externally.

• Approve the exact request.

• Reuse the code and show DENY.

• Change an argument and show the same code cannot approve it.

• Show every attempt in the log.

## Definition of done and tests

• Approval is a one-time authorization, not a general permission.

# 50. End-to-End Test D: Tamper and Replay

**Purpose:** Demonstrate evidence integrity.

**Work through this page in order.**

## Step-by-step procedure

- Copy original_log.jsonl to tampered_log.jsonl.
- Change one character in a payload using a text editor.
- Run verify against tampered log.
- Show affected line, sequence and event_id.
- Restore the original log.
- Run replay twice.
- Show identical decision digests.
- Explain that hash chaining detects modification but does not replace access control.

## Definition of done and tests

- The jury sees both a failure report for tampering and a successful replay of the original.

# 51. Final Jury Demo Script

**Purpose:** Turn the build into a clear five-minute story.

**Work through this page in order.**

## Step-by-step procedure

• Begin with the one-sentence problem: an AI can be manipulated into unsafe tool use.

• Show the seven rings as responsibilities.

• Show the MCP proxy as the mandatory gateway.

• Run malicious PDF and point to evidence ID.

• Run safe request and show ALLOW.

• Run HOLD with external approval.

• Tamper with log and verify.

• Replay original log and show same decisions.

• End with the principle: AI interprets, policy enforces, log proves.

## Definition of done and tests

• Every team member speaks for their module and can answer what happens when it fails.

# 52. Troubleshooting and Backup Plan

**Purpose:** Prepare for failures during the live round.

**Work through this page in order.**

## Step-by-step procedure

- If external notification fails, use the approved mock notification adapter and explain the adapter boundary.
- If the live model is unavailable, use recorded analyser output for the deterministic demo.
- If UI fails, run the CLI demo and show the evidence JSON.
- If a merge conflict occurs, use the last passing integration tag.
- If a log is corrupted, use the saved original fixture.
- If a tool hangs, use timeouts and record tool_timeout as a denial or containment event.
- Never improvise with real data or real external transfers.

## Definition of done and tests

- A backup path still demonstrates the required security properties.

# 53. Final Submission Checklist

**Purpose:** Package the project so the jury can run it.

**Work through this page in order.**

## Step-by-step procedure

• Push main with README, setup instructions and demo commands.

• Include requirements.txt and .env.example.

• Include tests and synthetic scenario fixtures.

• Include docs/architecture.md and docs/TEAM_IMPLEMENTATION_GUIDE.md.

• Remove secrets, real data and temporary logs.

• Verify a clean clone can run pytest.

• Verify the GitHub link is correct before submitting the form.

• Prepare screenshots or a short backup recording if permitted.

## Definition of done and tests

• The repository, PDF guide and live demo all tell the same story.

# 54. Definition of Done

**Purpose:** Use this page as the final sign-off.

**Work through this page in order.**

## Step-by-step procedure

• Every tool call passes through MCP proxy.

• Every decision is ALLOW, HOLD or DENY and has evidence.

• Every event is hash chained and verifiable.

• Tampering identifies the affected entry.

• Replay is deterministic.

• Malicious PDF is blocked or held.

• Safe request is allowed.

• Approval is exact, one-time and expiring.

• All four members have passing tests and can explain their code.

• Main branch is tagged and ready for the jury.

## Definition of done and tests

• Run pytest -q, run the four demo scenarios, run verify, run replay twice and archive the outputs in docs/demo-evidence/.